DAY 4 · MODULE 1 · 35 MIN

Agents vs. Workflows

Choose who decides what happens next, then compose across boundaries

Source: https://learn.microsoft.com/agent-framework/journey/workflows

Two philosophies, not two products

Agents decide

- The model picks which tool to call, whether to delegate, when to stop
- Powerful for open-ended tasks where the right path depends on the conversation

Workflows decide

- You define the graph explicitly: steps, order, decision points
- Essential when the process itself has rules

Source: https://learn.microsoft.com/agent-framework/journey/workflows

The intelligence spectrum

1 Single agent with tools

the model decides every step; most flexible, least predictable

2 Workflow with agent executors

the graph controls the process; agents reason at specific steps

3 Workflow with deterministic executors only

no LLM at all; a traditional, fully predictable pipeline

Source: https://learn.microsoft.com/agent-framework/journey/workflows

Choosing the right pattern

Question	If the model decides	If you decide
Which subtask to tackle next?	Agents as tools	Workflows
Whether to involve another agent?	Agents as tools	Agents in workflows
When to ask a human?	Tool approval (reactive, per-tool)	Human-in-the-loop (explicit gates)
How to handle partial failure?	Retry logic in tool implementations	Checkpoints

Source: https://learn.microsoft.com/agent-framework/journey/workflows

When workflows earn their complexity

- A document-review pipeline where a draft must be written, reviewed, revised, and approved — in that order, every time
- A customer-onboarding flow: collect info, run a compliance check, provision accounts, send a welcome email — some steps parallel, some gated by human approval
- An analytics workflow that should resume from the last checkpoint after a failure, not start over

Prefer simpler patterns first — reach for workflows when order, gates, or resumability genuinely matter.

Source: https://learn.microsoft.com/agent-framework/journey/workflows

Recap: agents as tools

- One agent (outer) calls another agent (inner) as if it were a regular function tool
- The outer agent decides *when* and *whether* to delegate — the same way it decides to call any tool
- The inner agent runs independently: its own instructions, tools, and model calls
- Limitation: the outer agent sees only the inner agent's final text response, not its reasoning or tool calls

This works well within one process, one team, one runtime. What about across a boundary?

Source: https://learn.microsoft.com/agent-framework/journey/agents-as-tools

Crossing a boundary: Agent-to-Agent (A2A)

- Agent-to-Agent (A2A) is an open protocol for agents to discover each other, exchange messages, and coordinate — over HTTP, in any language or framework
- Use it when you cross a boundary in-process composition can't handle:
  - Service boundaries — the other agent runs as its own microservice
  - Team boundaries — a partner team owns the agent; you don't have its code or model
  - Organizational boundaries — a third-party agent (document processing, legal review) needs a standard way to be discovered and called
  - Independent evolution — different release cycles, teams, or languages

If your agents share a process and a team, agents-as-tools is simpler — A2A adds value specifically at a boundary.

Source: https://learn.microsoft.com/agent-framework/journey/agent-to-agent

Call a remote agent over A2A

```python
from agent_framework.a2a import A2AAgent

async with A2AAgent(name="remote", url="https://a2a-agent.example.com") as agent:
    response = await agent.run("Hello!")
    print(response.messages[0].text)
```

A2AAgent ships in a separate, prerelease package: pip install agent-framework-a2a --pre.

Source: https://learn.microsoft.com/agent-framework/integrations/by-component/agent-services/a2a

Wrap a workflow as an agent

```
workflow = SequentialBuilder(participants=[researcher, writer]).build()

# Wrap the whole workflow behind the standard agent interface
workflow_agent = workflow.as_agent(name="Content Pipeline Agent")

response = await workflow_agent.run("Write an article about AI trends")
```

A caller — another agent, an A2A client, your own code — can't tell it's talking to a workflow instead of a single agent.

Source: https://learn.microsoft.com/agent-framework/workflows/as-agents

D E M O

~4 min

Wrap a workflow, call it like any other agent

Run the official sequential_workflow_as_agent.py sample live: a two-agent writer→reviewer pipeline wrapped as an agent returns exactly one message, matching the previous slide's code. Then run a comparison script that builds the SAME workflow with and without intermediate_output_from=[writer] — proving .as_agent() exposes only whatever output designation the workflow already had, nothing more.

Runbook: demos/day4/module-1-demo-1-workflow-as-agent.md

Source: https://github.com/microsoft/agent-framework/blob/main/python/samples/03-workflows/agents/sequential_workflow_as_agent.py

The composition circle

1 Agents as tools

compose in-process, model-driven delegation

2 Agent-to-Agent (A2A)

compose across a process/service/org boundary

3 Workflows

compose with an explicit, developer-defined graph (today's Modules 2-6)

4 Workflows as agents

wrap the whole graph back behind the standard agent interface

Source: https://learn.microsoft.com/agent-framework/journey/workflows

Takeaways

- You decide, per step, whether the model or the code should control what happens next.
- You use agents as tools for in-process, model-driven delegation.
- You reach for A2A specifically when you cross a process, service, or organizational boundary.
- You build workflows when the process itself has rules: fixed order, human gates, or resumability.
- You can wrap any workflow behind the standard agent interface with .as_agent() — composition goes full circle.

Source: https://learn.microsoft.com/agent-framework/journey/workflows · +1 in notes